Air University

Operating Systems (Lab)

Year 2026

Ms. Qurrat Ul Ain

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: May 15, 2026

Lab # 10

Environment Setup:

```
usama@ubuntu:~/Desktop$ mkdir Lab10
usama@ubuntu:~/Desktop$ cd Lab10
```

1. Same Arrival Code

Source Code:

```
usama@ubuntu:~/Desktop/Lab10$ nano same_arrival.c
```

```c
#include <stdio.h>
#include <string.h>

#define MAX 10

int main(void)
{
    int   n;
    int   pid[MAX], bt[MAX], at[MAX];
    int   wt[MAX], tat[MAX], ct[MAX];
    float avg_wt = 0, avg_tat = 0;

    /* — Input — */
    printf("=======================================\n");
    printf("   FCFS Scheduling (Same Arrival Time)  \n");
    printf("=======================================\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        pid[i] = i + 1;

        printf("  Burst Time  : ");
        scanf("%d", &bt[i]);

        at[i] = 0;   /* All arrive at time 0 */
    }

    /* — Calculate Waiting Time — */
    wt[0] = 0;   /* First process waits 0 ms */
    for (int i = 1; i < n; i++) {
        wt[i] = 0;
        for (int j = 0; j < i; j++)
            wt[i] += bt[j];
        wt[i] -= at[i];   /* at[i] == 0 so no real effect here */
        if (wt[i] < 0) wt[i] = 0;
    }
```

```c
    /* ── Calculate Turnaround Time and Completion Time ── */
    for (int i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        ct[i]  = at[i] + tat[i];
    }

    /* ── Averages ── */
    for (int i = 0; i < n; i++) {
        avg_wt  += wt[i];
        avg_tat += tat[i];
    }
    avg_wt  /= n;
    avg_tat /= n;

    /* ── Gantt Chart ── */
    printf("\n");
    printf("========================================\n");
    printf("              GANTT CHART               \n");
    printf("========================================\n");

    /* Top border */
    printf(" ");
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < bt[i] * 3; k++) printf("-");
        printf(" ");
    }
    printf("\n|");

    /* Process labels */
    for (int i = 0; i < n; i++) {
        int spaces = bt[i] * 3 - 1;
        int left   = spaces / 2;
        int right  = spaces - left;
        for (int k = 0; k < left;  k++) printf(" ");
        printf("P%d", pid[i]);
        for (int k = 0; k < right - 1; k++) printf(" ");
        printf("|");
    }
```

```c
    /* Bottom border */
    printf("\n ");
    for (int i = 0; i < n; i++) {
        for (int k = 0; k < bt[i] * 3; k++) printf("-");
        printf(" ");
    }

    /* Time markers */
    printf("\n0");
    int time = 0;
    for (int i = 0; i < n; i++) {
        time += bt[i];
        int spaces = bt[i] * 3 - 1;
        if (time < 10)
            printf("%*d", spaces + 1, time);
        else
            printf("%*d", spaces, time);
    }
    printf("\n");

    /* ── Results Table ── */
    printf("\n========================================\n");
    printf("%-5s %-5s %-5s %-5s %-5s %-5s\n",
           "PID", "AT", "BT", "CT", "TAT", "WT");
    printf("----------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-5d %-5d %-5d %-5d %-5d %-5d\n",
               pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    printf("========================================\n");
    printf("Average Waiting Time    : %.2f\n", avg_wt);
    printf("Average Turnaround Time : %.2f\n", avg_tat);
    printf("========================================\n");

    return 0;
}
```

Compilation & Permission:

```
usama@ubuntu:~/Desktop/Lab10$ gcc -Wall -o fcfs_same same_arrival.c
```

```
usama@ubuntu:~/Desktop/Lab10$ chmod +x fcfs_same
```

Output:

```
usama@ubuntu:~/Desktop/Lab10$ ./fcfs_same
========================================
   FCFS Scheduling (Same Arrival Time)
========================================
Enter number of processes: 4

Process 1:
  Burst Time  : 6

Process 2:
  Burst Time  : 5

Process 3:
  Burst Time  : 3

Process 4:
  Burst Time  : 4

========================================
             GANTT CHART
========================================
 ------------------ -------------- --------- ------------
|        P1        |       P2     |    P3   |     P4     |
 ------------------ -------------- --------- ------------
0                 6             11        14          18

========================================
PID   AT    BT    CT    TAT   WT
----------------------------------------
1     0     6     6     6     0
2     0     5     11    11    6
3     0     3     14    14    11
4     0     4     18    18    14
========================================
Average Waiting Time    : 7.75
Average Turnaround Time : 12.25
========================================
```

2. Different Arrival Code

Source Code

```
usama@ubuntu:~/Desktop/Lab10$ nano diff_arrival.c
```

```c
#include <stdio.h>
#include <string.h>

#define MAX 10

int main(void)
{
    int n;
    int pid[MAX], bt[MAX], at[MAX];
    int wt[MAX], tat[MAX], ct[MAX];
    int start[MAX];       /* start time of each process on CPU */
    float avg_wt = 0, avg_tat = 0;

    /* — Input — */
    printf("========================================\n");
    printf("  FCFS Scheduling (Different Arrival Time)\n");
    printf("========================================\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d:\n", i + 1);
        pid[i] = i + 1;
        printf("  Arrival Time : ");
        scanf("%d", &at[i]);
        printf("  Burst Time   : ");
        scanf("%d", &bt[i]);
    }

    /* — Sort by Arrival Time (Bubble Sort) — */
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                int tmp;
                tmp = at[j];  at[j] = at[j+1];  at[j+1] = tmp;
                tmp = bt[j];  bt[j] = bt[j+1];  bt[j+1] = tmp;
                tmp = pid[j]; pid[j]= pid[j+1]; pid[j+1]= tmp;
            }
        }
    }
```

```c
/* ── Calculate Start, Completion, WT, TAT ── */
/*
 * wt[i] = (sum of bt[0..i-1]) - at[i]
 * But we must account for CPU idle time when no process has arrived.
 * start[0] = at[0]
 * start[i] = max(ct[i-1], at[i])
 */
start[0] = at[0];
ct[0]    = start[0] + bt[0];
wt[0]    = 0;
tat[0]   = ct[0] - at[0];

for (int i = 1; i < n; i++) {
    /* CPU may be idle if next process hasn't arrived yet */
    start[i] = (ct[i-1] > at[i]) ? ct[i-1] : at[i];
    ct[i]    = start[i] + bt[i];
    wt[i]    = start[i] - at[i];      /* time spent waiting */
    if (wt[i] < 0) wt[i] = 0;
    tat[i]   = ct[i] - at[i];         /* total time in system */
}

/* ── Averages ── */
for (int i = 0; i < n; i++) {
    avg_wt  += wt[i];
    avg_tat += tat[i];
}
avg_wt  /= n;
avg_tat /= n;

/* ── Gantt Chart ── */
printf("\n");
printf("========================================\n");
printf("               GANTT CHART              \n");
printf("========================================\n");

/*
 * We draw blocks proportional to burst time (each unit = 3 chars).
 * If CPU is idle between processes, we draw an IDLE block.
 */

/* Build timeline: collect (label, duration) pairs */
```

```c
    /* Build timeline: collect (label, duration) pairs */
    char  labels[MAX * 2][6];
    int   durations[MAX * 2];
    int   seg = 0;
    int   cur_time = at[0];

    for (int i = 0; i < n; i++) {
        /* Idle gap? */
        if (start[i] > cur_time) {
            snprintf(labels[seg], sizeof(labels[seg]), "IDLE");
            durations[seg] = start[i] - cur_time;
            seg++;
        }
        snprintf(labels[seg], sizeof(labels[seg]), "P%d", pid[i]);
        durations[seg] = bt[i];
        seg++;
        cur_time = ct[i];
    }

    /* Top border */
    printf(" ");
    for (int s = 0; s < seg; s++) {
        for (int k = 0; k < durations[s] * 3; k++) printf("-");
        printf(" ");
    }
    printf("\n|");

    /* Labels */
    for (int s = 0; s < seg; s++) {
        int total  = durations[s] * 3 - 1;
        int llen   = (int)strlen(labels[s]);
        int pad    = total - llen;
        int left   = pad / 2;
        int right  = pad - left;
        for (int k = 0; k < left;  k++) printf(" ");
        printf("%s", labels[s]);
        for (int k = 0; k < right; k++) printf(" ");
        printf("|");
    }
```

```c
    /* Bottom border */
    printf("\n ");
    for (int s = 0; s < seg; s++) {
        for (int k = 0; k < durations[s] * 3; k++) printf("-");
        printf(" ");
    }

    /* Time markers */
    printf("\n%d", at[0]);
    int t = at[0];
    for (int s = 0; s < seg; s++) {
        t += durations[s];
        int spaces = durations[s] * 3 - 1;
        if (t < 10)
            printf("%*d", spaces + 1, t);
        else
            printf("%*d", spaces, t);
    }
    printf("\n");

    /* — Results Table — */
    printf("\n========================================\n");
    printf("%-5s %-5s %-5s %-6s %-6s %-5s %-5s\n",
           "PID", "AT", "BT", "Start", "CT", "TAT", "WT");
    printf("----------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-5d %-5d %-5d %-6d %-6d %-5d %-5d\n",
               pid[i], at[i], bt[i], start[i], ct[i], tat[i], wt[i]);
    }
    printf("========================================\n");
    printf("Average Waiting Time    : %.2f\n", avg_wt);
    printf("Average Turnaround Time : %.2f\n", avg_tat);
    printf("========================================\n");

    return 0;
}
```

Compilation & Permission

```
usama@ubuntu:~/Desktop/Lab10$ gcc -Wall -o fcfs_diff diff_arrival.c
```

```
usama@ubuntu:~/Desktop/Lab10$ chmod +x fcfs_diff
```

Output

```
usama@ubuntu:~/Desktop/Lab10$ ./fcfs_diff
==========================================
  FCFS Scheduling (Different Arrival Time)
==========================================
Enter number of processes: 4

Process 1:
  Arrival Time : 0
  Burst Time   : 4

Process 2:
  Arrival Time : 1
  Burst Time   : 3

Process 3:
  Arrival Time : 2
  Burst Time   : 1

Process 4:
  Arrival Time : 8
  Burst Time   : 2

==========================================
                GANTT CHART
==========================================
 ----------- --------- --- ------
|    P1     |   P2    |P3| P4  |
 ----------- --------- --- ------
0           4         7  8   10

==========================================
PID   AT    BT    Start  CT     TAT   WT
------------------------------------------
1     0     4     0      4      4     0
2     1     3     4      7      6     3
3     2     1     7      8      6     5
4     8     2     8      10     2     0
==========================================
Average Waiting Time    : 2.00
Average Turnaround Time : 4.50
==========================================
```

- End of Lab Task -